4.1.3. Exploration

Recall from the previous chapter that we needed our policy to include some randomness, which would allow us to visit new states during training. For DQNs we followed the epsilon-greedy policy, where there was a chance we would not follow the action that led to the greatest predicted reward. If we always selected the action that led to the maximum predicted reward, we'd never discover the even better actions and states available to us. For the stochastic policy gradient method, because our output is a probability distribution, there should be a small chance that we explore all spaces; only after sufficient exploration will the action distribution converge to producing the single best action, a degenerate distribution. Or if the environment itself has some randomness, the probability distribution will retain some probability mass to each action. When we initialize our model in the beginning, the probability of our agent picking each action should be approximately equal or uniform, since the model has zero information about which action is better.

There is a variant of policy gradient called *deterministic policy gradient* (DPG) where there is a single output that the agent will always follow (as illustrated in figure 4.4). In the case of Gridworld, for example, it would produce a 4-dimensional binary vector with a 1 for the action to be taken and 0s for the other actions. The agent won't explore properly if it always follows the output because there's no randomness in the action selection. Since the output of a deterministic policy function for a discrete action set would be discrete values, it is also difficult to get this working in the fully differentiable manner that we are accustomed to with deep learning, so we'll focus on stochastic policy gradients. Building a notion of uncertainty into the models (e.g., using probability distributions) is generally a good thing.

4.2. Reinforcing good actions: The policy gradient algorithm

From the previous section, you understand that there is a class of algorithms that attempts to create a function that outputs a probability distribution over actions, and that this policy function $\pi(s)$ can be implemented with a neural